



Programação Web

Padrão Model-View-Controller

*Node.js · Express · PostgreSQL · Arquitetura
em camadas*

Prof. Dr. Renato de Sousa Gomide

Objetivos da aula

- Entender **por que** separar responsabilidades em uma aplicação web
- Conhecer o padrão **MVC** e seus **três componentes**
- Identificar o papel de **Model**, **View** e **Controller** em um projeto Express
- Percorrer o **fluxo de dados** de uma requisição do cliente até a resposta
- Reconhecer os **benefícios** da separação: manutenção, reúso, testes
- Implementar um **CRUD completo** (Todo List) seguindo o padrão

O problema: tudo em um arquivo só

Nos módulos anteriores, uma rota fazia **tudo**:

```
app.get('/users', async (req, res) => {  
  const result = await pool.query('SELECT * FROM users'); // acesso a dados  
  const users = result.rows.map(u => ({ // formatação  
    id: u.id, username: u.username, email: mascarar(u.email)  
  }));  
  res.json(users); // resposta HTTP  
});
```

Acesso a dados + regra de negócio + formatação + HTTP no **mesmo lugar**.

Funciona com 1 rota. Com 40 rotas, vira **código espaguete**.

Analogia: o restaurante

Papel	No restaurante	Na aplicação
Garçom	Recebe o pedido, leva à cozinha, traz o prato	Controller
Cozinha	Prepara o prato, conhece os ingredientes	Model
Prato montado	Como a comida é apresentada ao cliente	View

O cliente **não entra na cozinha**. O cozinheiro **não atende mesas**.

Cada um faz **uma coisa** e faz bem. Trocar o cardápio não muda o jeito do garçom trabalhar.

O que é o MVC?

MVC é um padrão de arquitetura que separa a **lógica de negócios** da **interface do usuário**, dividindo a aplicação em três componentes:

Model

Dados e lógica de negócios.
Fala com o banco.

View

Apresentação dos dados ao usuário.

Controller

Coordena Model e View.
Recebe requisições.

Amplamente usado em desenvolvimento web — Rails, Django, Laravel, Spring MVC e, aqui, Express organizado à mão.

Stack do projeto



Node.js + Express



PostgreSQL (pg)



JavaScript (CommonJS)

Sem framework MVC: a estrutura é criada por **convenção de pastas** e **require**.

```
src/  
├── controller/user.controller.js  
├── model/user.model.js  
├── view/user.view.js  
├── db/index.js  
└── index.js
```

1. Model (Modelo)

- Representa os **dados** e a **lógica de negócios**
- Gerencia o **acesso aos dados** (banco, arquivos, APIs)
- Contém regras de **validação** e manipulação
- É **independente** da interface do usuário

src/model/user.model.js :

```
const { query } = require('../db');

const findAll = async () => {
  const users = await query('SELECT * FROM users');
  return users.rows;
};

module.exports = { findAll };
```

O Model não sabe quem o chama

O Model devolve **dados crus** (`rows` do PostgreSQL). Ele **não** sabe:

- se a resposta vai virar JSON, HTML ou CSV
- qual rota HTTP o chamou
- qual status code será usado

Consequência prática: o mesmo `findAll()` serve à API REST, a um relatório em PDF e a um script de linha de comando. **Reúso** vem da ignorância proposital.

2. View (Visão)

- Responsável pela **apresentação** dos dados
- Exibe as informações do Model
- Pode ser HTML, **JSON**, template, ou qualquer formato de saída
- **Não** contém lógica de negócios

src/view/user.view.js :

```
const toJson = (user) => {  
  const email = maskWithAsterisks(user.email, ['@', '.'],  
                                   [0, user.email?.length - 1]);  
  return { id: user.id, username: user.username, email };  
};  
  
const arrayToJson = (users) => users.map((user) => toJson(user));
```

View em uma API REST

Sem tela para desenhar, a View vira o **formato da resposta JSON**.

Repare no que a `user.view.js` faz:

Ação	Efeito
Escolhe os campos	<code>id</code> , <code>username</code> , <code>email</code> — <code>password</code> fica de fora
Mascara o e-mail	<code>j***@***.com</code>
<code>arrayToJson</code>	Aplica <code>toJson</code> a cada item da lista

O banco tem `password` . A View **decide não mostrar**. Segurança e apresentação no lugar certo — não espalhadas por cada rota.

VO — Value Object

Note o nome da variável no Controller: `usersVO` .

- **VO** (*Value Object*) = objeto criado só para **transportar dados** até o cliente
- Também chamado de **DTO** (*Data Transfer Object*)
- É a **saída da View**, não a linha do banco

Distinção mental útil: **entidade** (o que está no banco, com `password`) $\neq$ **VO** (o que sai na resposta, sem `password`).

3. Controller (Controlador)

- Recebe as **requisições** do usuário
- **Coordena** as interações entre Model e View
- Processa os dados recebidos
- Decide **qual View** será usada

3. Controller (Controlador)

src/controller/user.controller.js :

```
const router = require('express').Router();
const userModel = require('../model/user.model');
const userView = require('../view/user.view');

router.get('/users', async (request, response) => {
  const users = await userModel.findAll(); // 1. pede ao Model
  const usersV0 = userView.arrayToJson(users); // 2. formata na View
  response.json(usersV0); // 3. responde
});

module.exports = router;
```

O Controller é magro

Três linhas no handler. Isso é **proposital**.

Regra de bolso: se o Controller tem `SELECT`, ele está fazendo trabalho de Model. Se tem `map` montando campos de resposta, está fazendo trabalho de View.

O Controller decide **o que** chamar e **em que ordem** — não **como** as coisas funcionam.

`express.Router()` — o Controller como módulo

Cada Controller exporta um **router** próprio, montado no `index.js`:

```
const express = require('express');
const app = express();
const userController = require('./controller/user.controller');

app.use(express.json());
app.use("/", userController);

app.listen(3000, () => {
  console.log('Servidor escutando na porta 3000');
});
```

Adicionar um recurso novo = criar um arquivo de controller + **uma linha** de `app.use`. O `index.js` não cresce junto com a aplicação.

A camada de acesso ao banco

`src/db/index.js` isola o **pool de conexões** do PostgreSQL:

```
const { Pool } = require('pg');

const pool = new Pool({
  host: process.env.DB_HOST, port: process.env.DB_PORT,
  user: process.env.DB_USER, password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
});

const query = async (text, params) => await pool.query(text, params);

module.exports = { closeConnection, query };
```

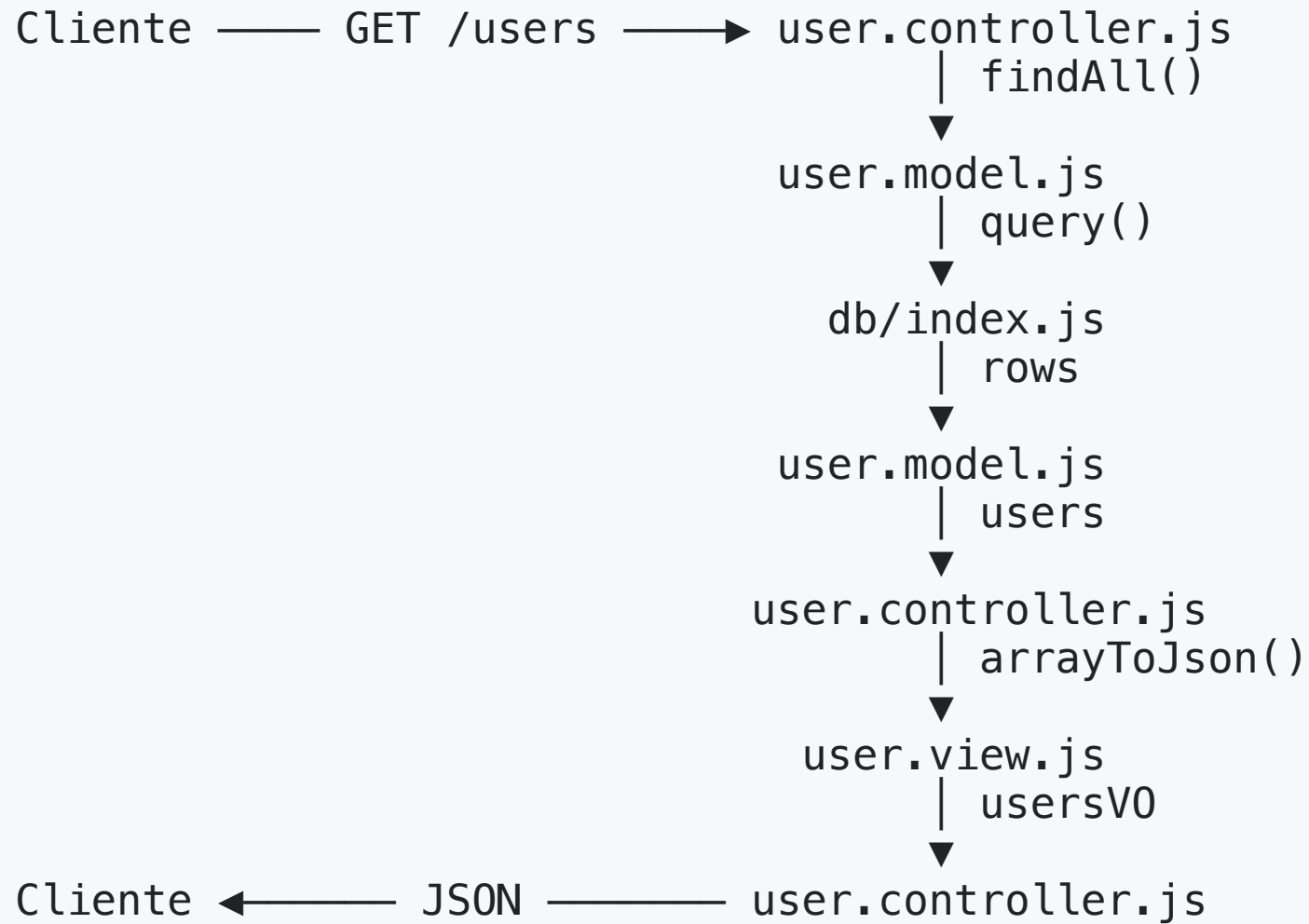
Só o **Model** importa o `db`. Controller e View **nunca** conhecem SQL.

Fluxo de dados no MVC

1. O usuário faz uma requisição
2. O **Controller** recebe a requisição
3. O Controller solicita dados ao **Model**
4. O Model processa os dados e retorna ao Controller
5. O Controller envia os dados para a **View**
6. A View formata os dados e retorna ao Controller
7. O Controller envia os dados para o usuário

O Controller é o **único** que fala com todo mundo. Model e View **não se conhecem**.

Fluxo de GET /users



Quem pode chamar quem?

De → Para	Permitido?	Por quê
Controller → Model	Sim	Pedir/gravar dados
Controller → View	Sim	Formatar a resposta
Model → db	Sim	Único acesso ao banco
Model → View	Não	Model não sabe como será exibido
View → Model	Não	View não busca dados sozinha
Controller → db	Não	Pula a camada de Model

Violar essas setas é o começo do acoplamento que o MVC existe para evitar.

Benefícios do MVC

1. Separação de responsabilidades

- cada componente tem uma função
- facilita manutenção e trabalho em equipe

2. Reutilização de código

- Models servem a várias Views
- Views servem a vários Models

3. Manutenibilidade

- código organizado, novas funcionalidades e correções mais simples

4. Testabilidade

- componentes isolados são mais fáceis de testar unitariamente

O custo do padrão

Honestidade intelectual: MVC **não é grátis**.

- Mais **arquivos** e mais `require` para uma rota trivial
- Camada de indireção: ler o código exige pular entre 3 arquivos
- Para um script de 50 linhas, é **excesso de estrutura**

O ganho aparece com **escala** e **tempo de vida** do projeto. Trocar PostgreSQL por MongoDB mexe só no Model; mudar o formato da resposta mexe só na View.

MVC em outros contextos

Contexto	View típica
API REST (este projeto)	JSON serializado
Web tradicional	Template HTML (EJS, Handlebars)
Desktop / Mobile	Componentes de tela

Variações comuns do padrão: **MVP** (*Model-View-Presenter*), **MVVM** (*Model-View-ViewModel*), **Clean Architecture**.

A ideia central sobrevive em todas: **quem guarda os dados** ≠ **quem apresenta** ≠ **quem coordena**.

Exercício — Todo List em MVC

Implementar um sistema de gerenciamento de tarefas seguindo o padrão, integrado ao projeto existente.

1. Banco de dados — `sql/todo.sql` :

```
create table todos (  
  id serial primary key,  
  title varchar(255) not null,  
  description text,  
  completed boolean default false,  
  user_id integer references users(id),  
  created_at timestamp default current_timestamp  
);
```

Exercício — 2. Model

src/model/todo.model.js :

```
const { query } = require('../db');

const findAll = async () => {
  const todos = await query('SELECT * FROM todos ORDER BY created_at DESC');
  return todos.rows;
};

const findById = async (id) => {
  const todo = await query('SELECT * FROM todos WHERE id = $1', [id]);
  return todo.rows[0];
};

module.exports = { findAll, findById, create, update, remove };
```

`$1`, `$2` são **placeholders** do `pg` — protegem contra SQL Injection.

Exercício — 2. Model (escrita)

```
const create = async (todo) => {
  const { title, description, user_id } = todo;
  const result = await query(
    'INSERT INTO todos (title, description, user_id) VALUES ($1, $2, $3) RETURNING *',
    [title, description, user_id]
  );
  return result.rows[0];
};

const update = async (id, todo) => {
  const { title, description, completed } = todo;
  const result = await query(
    'UPDATE todos SET title = $1, description = $2, completed = $3 WHERE id = $4 RETURNING *',
    [title, description, completed, id]
  );
  return result.rows[0];
};

const remove = async (id) => {
  await query('DELETE FROM todos WHERE id = $1', [id]);
};
```

Exercício — 3. View

src/view/todo.view.js :

```
const toJson = (todo) => {  
  return {  
    id: todo.id,  
    title: todo.title,  
    description: todo.description,  
    completed: todo.completed,  
    user_id: todo.user_id,  
    created_at: todo.created_at  
  };  
};  
  
const arrayToJson = (todos) => todos.map(todo => toJson(todo));  
  
module.exports = { toJson, arrayToJson };
```

Exercício — 4. Controller (leitura)

src/controller/todo.controller.js :

```
const router = require('express').Router();
const todoModel = require('../model/todo.model');
const todoView = require('../view/todo.view');

router.get('/todos', async (request, response) => {
  const todos = await todoModel.findAll();
  const todosV0 = todoView.arrayToJson(todos);
  response.json(todosV0);
});

// continua no próximo slide...
```

Exercício — 4. Controller (leitura)

```
router.get('/todos/:id', async (request, response) => {  
  const { id } = request.params;  
  const todo = await todoModel.findById(id);  
  
  if (!todo) {  
    return response.status(404).json({ error: 'Tarefa não encontrada' });  
  }  
  response.json(todoView.toJson(todo));  
});
```

Exercício — 4. Controller (escrita)

```
router.post('/todos', async (request, response) => {
  const todo = request.body;
  const newTodo = await todoModel.create(todo);
  response.status(201).json(todoView.toJson(newTodo));
});

router.put('/todos/:id', async (request, response) => {
  const { id } = request.params;
  const updatedTodo = await todoModel.update(id, request.body);

  if (!updatedTodo) {
    return response.status(404).json({ error: 'Tarefa não encontrada' });
  }
  response.json(todoView.toJson(updatedTodo));
});

router.delete('/todos/:id', async (request, response) => {
  await todoModel.remove(request.params.id);
  response.status(204).send();
});
```

Exercício — 5. Integração

src/index.js :

```
const express = require('express');
const app = express();
const userController = require('./controller/user.controller');
const todoController = require('./controller/todo.controller');

app.use(express.json());

app.use("/", userController);
app.use("/", todoController);

app.listen(3000, () => {
  console.log('Servidor escutando na porta 3000');
});
```

Uma linha por Controller. O padrão se paga aqui.

Exercício — 6. Endpoints para testar

Método	URL	Corpo	Resposta
GET	/todos	—	Lista de tarefas
GET	/todos/:id	—	Tarefa ou 404
POST	/todos	title, description, user_id	201 + tarefa criada
PUT	/todos/:id	title, description, completed	Tarefa atualizada
DELETE	/todos/:id	—	204 No Content

```
{ "title": "Minha tarefa", "description": "Descrição da tarefa", "user_id": 1 }
```

Exercício — 7. Desafios adicionais

1. **Validação de dados** nas requisições:
 - título é **obrigatório**
 - o **usuário existe**?
 - o **status** é válido?
2. **Filtros na listagem** de tarefas (por status, por usuário) usando **query params**

Pergunta de projeto: validação é responsabilidade do **Model** (regra de negócio) ou do **Controller** (dados da requisição)? Justifique sua escolha.

Resumo

- **MVC** separa **dados**, **apresentação** e **coordenação**
- **Model** — dados e regras; único que conhece o banco
- **View** — formata a saída (**VO**); esconde campos sensíveis
- **Controller** — recebe a requisição, chama Model, chama View, responde
- Fluxo: **Cliente → Controller → Model → Controller → View → Controller → Cliente**
- Model e View **não se conhecem**; o Controller é o intermediário

Próximos passos

- Camada de **Service** entre Controller e Model para regras complexas
- **Repository pattern** — abstrair a fonte de dados do Model
- Tratamento centralizado de **erros** com middleware
- **Testes unitários** de Model e View isolados

Referências

- MVC — Martin Fowler, *Patterns of Enterprise Application Architecture*
- Express — Router
- node-postgres (`pg`)
- `README.md` deste módulo — teoria e enunciado completo do exercício